Module Interface Specification for Physiotherapy Companion

Team 11, technically functional
Matthew Huynh
Cieran Diebolt
Vaisnavi Shanthamoorthy
Maham Siddiqui
Eman Ashraf

November 14, 2025

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [https://github.com/M9Huynh/technically-functional/blob/main/docs/SRS/S
—SS]

[Also add any additional symbols, abbreviations or acronyms —SS]

Contents

3 Introduction

The following document details the Module Interface Specifications for a Physiotherapy Companion app. The application is designed to identify users using computer vision for their prescribed physiotherapy exercise. In addition, an analysis will be performed on the user's movement and, after completion, the app will provide metrics to the user on their performance.

This tool utilizes OpenCV and MediaPipe for image recognition and then performs measurements on the number of repetitions, range of motion and perceived exertion.

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at https://github.com/M9Huynh/technically-functional.

4 Notation

The structure of the MIS for modules comes from **?**, with the addition that template modules have been adapted from **?**. The mathematical notation comes from Chapter 3 of **?**. For instance, the symbol := is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1|c_2 \Rightarrow r_2|...|c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Physiotherapy Companion.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in (-∞, ∞)
natural number	$\mathbb{N}$	a number without a fractional component in [1, ∞)
real	$\mathbb{R}$	any number in (-∞, ∞)
float	F	floating point numbers within (approx.) (-$1.798{*}10^{308}$, $1.798{*}10^{308}$)
object	X	an abstract representation of a collection of the above, similar to tuple
array	a	a collection of a single data type, n entries of the form (a_0, a_1, a_2, ... ,a_{n-1})
list	L	an array of non-fixed length

The specification of Physiotherapy Companion uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types.

In addition, Physiotherapy Companion uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	**Level 2**
Hardware-Hiding Module	
Behaviour-Hiding Module	User Interface Module Home Module Main Module Draw Module
Software Decision Module	Database Management Module Generator Module Metrics Module Analyze Module Feedback Module

Table 1: Module Hierarchy

6 Variables

Name	Type	Description
Name	String	This is the name of the user.
role	String	Values can be PT or patient
email	String	This will be used as their login ID
password	String	This is set by the user for login purposes
birthday	String	This is the user's birthday that will be used for verification purposes
license_no	Int	This is used for verification of PT
invite_code	String	This is given by the PT to the patient to allow registration on the application

Table 2: User Account Data Fields

7 MIS

7.1 Module - Account Management

ADT

This module's purpose is to initialize an instance of UserAccount and to handle any account-related tasks (e.g. Account creation (Patient), Account deletion (PT)). It is an adapter that links to the user_info_database and storage_database.

7.1.1 Uses

- name
- role
- email
- password
- birthday
- license_no
- invite_code

7.1.2 Syntax

Exported Constants

- *UserAccount_P*: ADT
- *UserAccount_PT*: ADT

Exported Access Programs

Access Routine Name	Input	Output	Exceptions
account_create	*Name* *Role* *Birthday* *License_no/* *Invite_code* *Email* *Password*	UserAccount_P or UserAccount_PT	Input_format_invalid Required_field_empty Account_already_ exists Incorrect_creds Invite_code_expired
account_delete	*Name* *email*	NA	required_field_empty patient_not_found
username_pw_match	*Email* *Password*	NA	wrong_email_ password
get_profile	*Name* *Birthday*	UserAccount	wrong_email_ password
add_analysis_to_ storagedb	*UserAccount_P* *Recent_analysis*	NA	User_not_found Upload_error
add_video_to_ storagedb	*UserAccount_P* *Recent_video*	NA	User_not_found Upload_error Video_too_short
get_storagedb_ analysis	*UserAccount_P*	Recent_video	User_not_found Download_error
get_userdb_info	*Email*	UserAccount_P OR UserAccount_PT	User_not_found Download_error

Table 3: Account Management Access Routines

7.1.3 Semantics

State Variables
A: an instance of UserAccount
B: an instance of UserAccount

State Invariant
$\forall A, B \in UserDatabase : A[i] \neq B[j]$
$0 < A < length(A) \cap A[i] \notin \emptyset$

7.1.4 Assumptions

7.1.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.1.6 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

7.2 Module - Dashboard

ADT

This module's purpose is to initialize an instance of UserAccount and to handle any account-related tasks (e.g. Account creation (Patient), Account deletion (PT)). It is an adapter that links to the user_info_database and storage_database.

7.2.1 Uses

- name
- role
- email
- password
- birthday
- license_no
- invite_code

7.2.2 Syntax

Exported Constants

- *UserAccount_Patient*: ADT
- *UserAccount_PT*: ADT

Exported Access Programs

7.2.3 Semantics

State Variables
A: an instance of UserAccount
B: an instance of UserAccount

State Invariant
$\forall A, B \in UserDatabase : A[i] \neq B[j]$
$0 < A < length(A) \cap A[i] \notin \emptyset$

7.2.4 Assumptions

7.2.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

7.2.6 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL http://citeseer.ist.psu.edu/428727.html.

8 Appendix

[Extra information if required —SS]

Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?
5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)
6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO_Explores)

Group - Reflection

1. N/A
2. N/A
3. N/A

4. I think that our other documents such as the requirements, hazard analysis need to be slightly adjusted as a result of the design documents. With the requirements specifically some of them pertain to how the application design is and after completing our initial draft of the MG and MIS, the design does not necessarily line up with the requirements document. Further, some of the hazards considered should be changed to include a couple more areas for hazards to occur. Lastly for both of the documents, the assumptions should be changed to incorporated design changes that we had not considered during that deliverable.

5. TODO

6. TODO

Matthew - Reflection

1. I think something that went well was our changes to overall workflow. One thing that we recognized was starting our document on google docs and then transferring our to sections to GitHub on the last two days the the deliverable was due led to some things going unchecked and the deliverable not turning out the way we had planned. Further it had also resulted in a 'panic' when we had to resolve merge conflicts between our sections soon to the deadline. Overall, we cut google docs for this deliverable and mainly focused on our sections within GitHub and doing multiple small PRs within the deliverable timeline and that had gone quite well. There are a few days left but a majority of the document has at least been accounted for and hopefully this workflow works for our team.

2. I think one pain point I experienced was time management. I had suggested that since the computing team's workload was quite different to the documentation team's, we could take on small sections within the document with hopes of submitting it early into the project. This did partially go well but with focus split in two areas it was hard to focus on the development side of things for the POC demo. This may be something that improves with time but as of now is currently a work in progress as development on my end is not where I want it to be.

3. I think that our design decisions partially came from speaking with our stakeholder (Kevin Yu) but other decisions were made with typical application creation. For example, the high-level was talked through with Kevin to simulate the timing and feedback delivery that you would encounter during physiotherapy but things like reporting, logging-in as well as account creation were from looking at other applications and aspects that went well from that end.

4. Group Q

5. Group Q

6. Group Q

Vaisnavi - Reflection

1. Something that went well was how we worked on creating more constant pull requests (PRs) to avoid merge conflicts. This worked out well with everyone consistently pushing more PRs early on, allowing someone on the team to review and effectively merge it in. Overall, the organization and collaboration went very smoothly, and it allowed us to ultimately finish ahead of our internal deadline so we had enough time for editing and formatting.

2. I think one of the major pain points through this deliverable was trying to balance both the development side for the POC demo while also trying to contribute to the document. As the main sub-team for the POC demo, we did take on smaller sections, but it was difficult to find that balance in time for both at the start.

3. We believe that most of our major design decisions stemmed directly from our meetings with our stakeholder, Kevin Yu. Throughout the development of this deliverable, a number of ideas that we originally planned had to be changed or completely re-worked based on the feedback we received. In several cases, Kevin helped us recognize that certain features were either not feasible within the course timeline or did not align with the true purpose and scope of our project. This was extremely helpful overall to have this input to tackle these decisions early on in the process.

4. Group Q

5. Group Q

6. Group Q